PRUEBA TÉCNICA · INGENIERO DE CALIDAD QA

# Farmatodo — Respuesta completa a los cuatro ejercicios

**Ing. Yoel Maximiliano Almirón** · 23 de octubre de 2026

| | |
|---|---|
| **Repositorio** | github.com/yoelalmiron1997/farmatodo-qa-test-engineering |
| **Portal de calidad** | yoelalmiron1997.github.io/farmatodo-qa-test-engineering (resultados reales de cada corrida) |
| **Última ejecución** | functional-tests #39992999223 · resultados en el portal → Test Execution |
| **Ejecuciones** | GitHub Actions → functional-tests (reporte resumen y evidencias por corrida) |

## Resumen de la entrega

| # | Ejercicio | Respuesta | Pág. |
|---|---|---|---|
| 1 | Suite automatizada de wordcounter.net + reporte resumen | Selenium + Pytest con Page Object Model. 20 casos E2E validan palabras, caracteres y top 3 de Keyword Density contra un oráculo independiente. Última corrida en CI: **16/16 casos funcionales en PASS**, 0 flaky; los 4 FAIL son los defectos del Ej. 2, reproducidos a propósito. | 2 |
| 2 | Reporte de fallas de WordCounter | **4 defectos confirmados** (1 mayor, 3 menores) y 2 oportunidades de mejora, con pasos, resultado esperado/actual y evidencia. | 4 |
| 3 | 3 problemas de UX en Farmatodo | **3 hallazgos** con vista, pasos para replicar, evidencia y propuesta. El más grave: entre 768 y 991 px el detalle de producto no muestra precio ni botón de compra. | 7 |
| 4 | Contador de frecuencia de palabras | Algoritmo propio en Python (solo biblioteca estándar). Salida **idéntica** al ejemplo del PDF. Complejidad O(N + U log U) en tiempo y O(U + L) en espacio. | 10 |

> **Cómo está resuelto.** Cada caso de prueba se especificó primero (requisito, riesgo, prioridad P0/P1/P2, pasos y resultado esperado) y después se automatizó. GitHub Actions ejecuta todo en cada cambio, guarda el historial de corridas y publica el portal. Los tests que fallan y luego pasan se marcan como *flaky*, nunca como aprobados, y una caída del sitio se registra como *BLOCKED*, no como fallo.

Todo lo que muestra este documento está en el repositorio: código, especificaciones ( `specs/` ), capturas ( `docs/assets/` ) y el historial de ejecuciones (rama `qa-history` ).

# Ejercicio 1 — Suite automatizada de wordcounter.net

**Ver en vivo:** Casos de prueba · Ejecución de la suite · Flaky tests · Trazabilidad · Corrida en GitHub Actions

## Qué valida

Los tres objetivos del enunciado, cada uno con un caso P0 y casos de borde:

1. **Conteo de palabras** (contador del panel *Details* y encabezado).
2. **Conteo de caracteres**.
3. **Top 3 de Keyword Density**: palabra y cantidad de ocurrencias, en orden.

## Cómo está construida

- **Selenium WebDriver + Pytest**, Chrome headless en CI. Selenium Manager resuelve el driver.
- **Page Object Model**: `WordCounterPage` y `KeywordDensityPanel` encapsulan localizadores y acciones, sin aserciones. Los valores del panel se leen en una sola llamada para evitar lecturas a medio redibujar.
- **Oráculo independiente**: el resultado esperado se calcula desde el texto de entrada con las reglas del sitio (relevadas en un spike de 42 observaciones reales), nunca copiando lo que muestra la página.
- **Aislamiento**: navegador y perfil nuevos por test, orden aleatorio y esperas explícitas (sin `sleep`).
- **Robustez**: hasta 2 reintentos en E2E; si pasa al reintentar queda *FLAKY*. Si el sitio no responde, *BLOCKED*. Cada fallo guarda captura y DOM.

## Casos y resultado de la última corrida en CI

functional-tests #39992999223 · 2026-10-23 17:11 UTC · Ubuntu 24.04 · Chrome 155 headless · Python 3.12 · Selenium 4.51

| Caso | Escenario | Prio. | Resultado |
|---|---|---|---|
| TC-WC-001 | Conteo básico de palabras | P0 | PASS |
| TC-WC-002 | Conteo básico de caracteres | P0 | PASS |
| TC-WC-003 | Top 3 de Keyword Density sin empates | P0 | PASS |
| TC-WC-004 | Texto de ejemplo del PDF en WordCounter | P1 | PASS |
| TC-WC-005 | Espacios adicionales, iniciales y finales | P1 | PASS |
| TC-WC-006 | Signos de puntuación | P1 | PASS |
| TC-WC-007 | Mayúsculas y minúsculas en Keyword Density | P1 | PASS |
| TC-WC-008 | Actualización del contenido | P1 | PASS |
| TC-WC-009 | Borrado total del texto | P1 | PASS |
| TC-WC-010 | Texto multilínea | P1 | PASS |
| TC-WC-011 | Empates en el top 3 de Keyword Density | P2 | PASS |
| TC-WC-012 | Acentos y ñ (texto en español) | P1 | PASS |
| TC-WC-013 | Números, guiones y apóstrofes | P2 | PASS |
| TC-WC-014 | Texto extenso | P2 | PASS |

| | | | |
|---|---|---|---|
| TC-WC-015 | Keyword Density con *stop words* | P1 | PASS |
| TC-WC-016 | Emojis y caracteres fuera del BMP | P2 | PASS |
| TC-WC-017 | Reproducción BUG-WC-001: punto sin espacio | P2 | FAIL conocido |
| TC-WC-018 | Reproducción BUG-WC-002: palabra formada solo por guiones | P2 | FAIL conocido |
| TC-WC-019 | Reproducción BUG-WC-003: espacio no separable | P2 | FAIL conocido |
| TC-WC-020 | Reproducción BUG-WC-004: emojis en caracteres | P2 | FAIL conocido |

TC-WC-017…020 describen el comportamiento correcto y fallan mientras el defecto exista. Cuando WordCounter los corrija, pasarán solos.

### Reporte resumen

Cada corrida genera el reporte en la página de la ejecución de GitHub Actions y en el artifact `functional-reports` (PDF, HTML y Markdown, con capturas y DOM de cada fallo). Corrida completa (43 casos: 20 E2E + algoritmo y framework):

| 43 | 39 | 0 | 4 | 0 | 0 |
|---|---|---|---|---|---|
| Ejecutados | Aprobados | Flaky | Fallidos (conocidos) | Omitidos | Bloqueados |

| Caso | Categoría | Descripción del error |
|---|---|---|
| TC-WC-017 | assertion | Keyword Density lista 2 entradas para `hola.mundo`, pero el contador indica 1 palabra (BUG-WC-001) |
| TC-WC-018 | assertion | `---` aparece en Keyword Density con 0 palabras contadas (BUG-WC-002) |
| TC-WC-019 | assertion | Keyword Density muestra `uno dos` como una sola entrada cuando el espacio es NBSP (BUG-WC-003) |
| TC-WC-020 | assertion | 16 caracteres en lugar de 14: cada emoji suma 2 (BUG-WC-004) |

**Ver en vivo:** Quality Dashboard · Test Execution · Reporte resumen y artifact functional-reports

**Quality gate: WARN.** Los cuatro fallos son defectos conocidos y documentados, así que el pipeline no se bloquea pero los deja visibles. Un fallo nuevo en un P0, o un P0 flaky, sí bloquearía.

### Cómo ejecutarla

```
git clone https://github.com/yoelalmiron1997/farmatodo-qa-test-engineering.git
cd farmatodo-qa-test-engineering
uv sync --group dev
uv run pytest tests/e2e --reruns 2 --qa-results-dir reports
uv run qa results build --results-dir reports --out reports/run-result.json
uv run qa report summary --run reports/run-result.json --out-dir reports
```

# Ejercicio 2 — Reporte de fallas de wordcounter.net

**Ver en vivo:** Reporte de fallas en el portal · Spike de WordCounter

Exploración funcional guiada por charters, sobre la configuración por defecto. Solo se reportan hallazgos reproducidos al menos dos veces y con evidencia. Cada defecto tiene además un test automatizado que lo reproduce.

Entorno: Windows 10 · Chromium 152.0.7977.167 · viewport 808 × 912 · 23/10/2026. Severidad: **Mayor** = distorsiona el resultado principal en un uso frecuente; **Menor** = resultado incorrecto en un caso poco frecuente.

| ID | Tipo | Severidad | Título |
|---|---|---|---|
| BUG-WC-001 | Defecto | Menor | Contador y Keyword Density no coinciden ante un punto sin espacio |
| BUG-WC-002 | Defecto | Menor | Un token formado solo por guiones aparece como palabra clave con 0 palabras |
| BUG-WC-003 | Defecto | **Mayor** | Keyword Density no separa palabras unidas por espacio no separable (NBSP) |
| BUG-WC-004 | Defecto | Menor | Cada emoji cuenta como 2 caracteres |
| OBS-WC-001 | Mejora | — | Lista de *stop words* incompleta e inconsistente |
| OBS-WC-002 | Mejora | — | La misma palabra en NFC y NFD se lista dos veces |

## BUG-WC-001 — Contador y Keyword Density no coinciden ante un punto sin espacio

Severidad: Menor · Prioridad: media · Caso automatizado: TC-WC-017

### Pasos

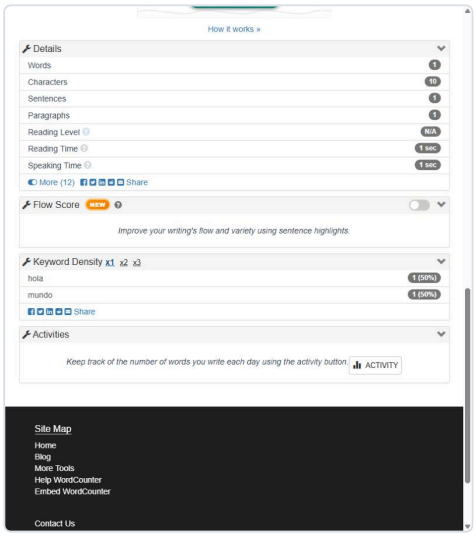
1. Abrir https://wordcounter.net/ con el editor vacío.
2. Escribir `hola.mundo` (sin espacio después del punto).
3. Observar el encabezado, *Details* y *Keyword Density*.

### Resultado esperado

Ambas funciones aplican la misma regla: si hay 1 palabra, Keyword Density muestra 1 entrada.

### Resultado actual

El contador indica **1 word**, pero Keyword Density lista **2** entradas: `hola` y `mundo`. Además, `hola,mundo` cuenta 2 palabras y `hola.mundo` cuenta 1.



1 palabra en Details; hola y mundo en Keyword Density

### Impacto y sugerencia

Omitir el espacio después del punto es un error de tipeo frecuente; los dos indicadores se contradicen. Unificar la tokenización de ambas funciones.

## BUG-WC-002 — Un token formado solo por guiones aparece como palabra clave con 0 palabras

Severidad: Menor · Prioridad: baja · Caso automatizado: TC-WC-018

### Pasos

1. Abrir https://wordcounter.net/ con el editor vacío.
2. Escribir `---`.
3. Observar el encabezado y *Keyword Density*.

### Resultado esperado

Con 0 palabras, Keyword Density no muestra entradas (como ocurre con números o letras sueltas).

### Resultado actual

El encabezado muestra **0 words 3 characters** y Keyword Density lista `---` con **1 (100 %)**.

### Impacto y sugerencia

Los separadores de texto (`---`, `***`) contaminan el ranking y pueden desplazar palabras reales del top. Excluir tokens sin letras ni dígitos.



0 palabras y “---” al 100 %

## BUG-WC-003 — Keyword Density no separa palabras unidas por espacio no separable (NBSP)

Severidad: **Mayor** · Prioridad: alta · Caso automatizado: TC-WC-019

### Pasos

1. Abrir https://wordcounter.net/.
2. Pegar `uno dos`, donde el espacio es U+00A0 (habitual al copiar desde Word o páginas web).
3. Observar *Details* y *Keyword Density*.

### Resultado esperado

Keyword Density muestra `uno` 1 y `dos` 1, coherente con el contador.

### Resultado actual

El contador indica **2 words**, pero Keyword Density muestra una única entrada **`uno dos` 1 (100 %)**.

### Impacto y sugerencia

Altera el resultado principal de Keyword Density al pegar texto: el top 3 puede mostrar pares inexistentes y ocultar las palabras reales. Tratar U+00A0 como espacio al tokenizar.



2 palabras; “uno dos” como una sola entrada

## BUG-WC-004 — Cada emoji cuenta como 2 caracteres

Severidad: Menor · Prioridad: baja · Casos: TC-WC-020 y TC-WC-016

### Pasos

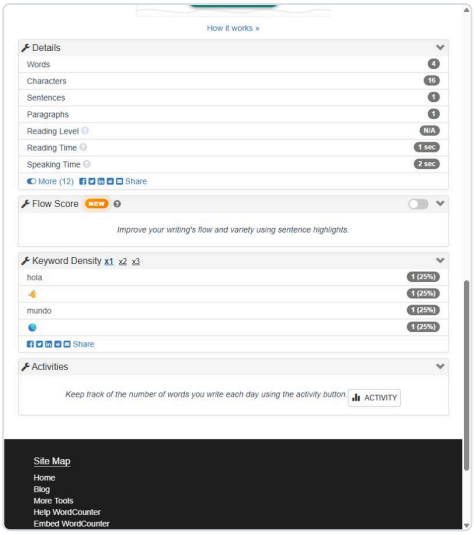
1. Abrir https://wordcounter.net/.
2. Pegar `hola 👋 mundo 🌎`.
3. Observar el contador de caracteres.

### Resultado esperado

**14** caracteres: 9 letras, 3 espacios y 2 emojis.

### Resultado actual

**16** caracteres: el sitio cuenta unidades UTF-16 (`String.length`), así que cada emoji suma 2. Además, los emojis aparecen en Keyword Density.



16 caracteres; emojis en Keyword Density

### Impacto y sugerencia

Quien escribe para un límite (redes sociales, SMS, SEO) ve un total mayor al real. Contar caracteres percibidos con `Intl.Segmenter`.

## Oportunidades de mejora

| ID | Observación | Sugerencia |
|---|---|---|
| OBS-WC-001 | Con `should would could`, Keyword Density excluye `would` pero lista `should` y `could`. Las *stop words* en español (`el`, `la`, `de`, `que`) nunca se excluyen. | Documentar y completar la lista, y aplicar la del idioma del texto. |
| OBS-WC-002 | `café` en forma NFC y en NFD (frecuente al copiar desde macOS) aparece como dos entradas distintas. | Normalizar a NFC antes de contar (el algoritmo del Ej. 4 ya lo hace). |

# Ejercicio 3 — Experiencia de usuario en Farmatodo

**Ver en vivo:** Hallazgos UX en el portal · UX-FT-001 · UX-FT-002 · UX-FT-003

**Método:** evaluación heurística (10 heurísticas de Nielsen y criterios de WCAG 2.2) sobre búsqueda, detalle de producto, carrito y checkout. Cada hallazgo se reprodujo y se verificó en el DOM. Severidad según la escala de Nielsen (0–4).

23/10/2026 · `www.farmatodo.com` es un selector de país, por eso el análisis se hizo en `www.farmatodo.com.co` (Colombia). Vistas: desktop a 1366 px, ventana de ~800 px y emulación móvil de 375 px. El checkout exige iniciar sesión: el análisis llega hasta ese paso, sin crear cuentas ni comprar.

| ID | Vista | Flujo | Problema | Severidad |
|---|---|---|---|---|
| UX-FT-001 | Web desktop, ventana de 768–991 px | Detalle de producto | No se muestran ni el precio ni el botón de compra | **4 — catastrófico** |
| UX-FT-002 | Web desktop (~800 px) | Búsqueda | Los filtros no se reconocen como seleccionables y no funcionan con teclado | 3 — mayor |
| UX-FT-003 | Web desktop (1366 y ~800 px) | Carrito y checkout | Se agrega un medicamento con fórmula sin aviso y una bolsa que el usuario no pidió | 3 — mayor |

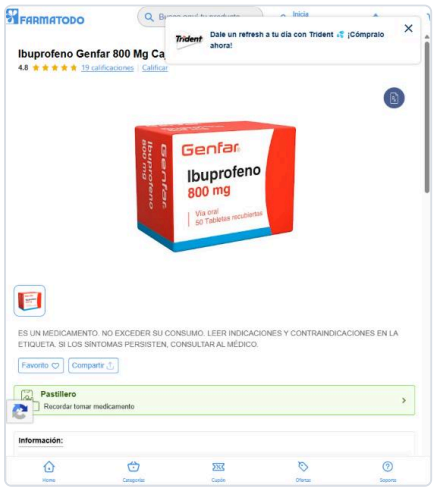
## UX-FT-001 — El detalle de producto no muestra precio ni botón de compra entre 768 y 991 px

Vista: web desktop con ventana de 768–991 px (pantalla dividida, laptop chica, zoom alto) · Severidad 4 — catastrófico para ese segmento

**Qué pasa.** En ese rango, la página muestra imagen, descripción y ficha técnica, pero **no el precio ni el botón "Agregar producto"**. En móvil existe una barra fija de compra y desde 992 px aparece el bloque de escritorio; en el medio no aparece ninguno.

**Cómo replicarlo.** Abrir `farmatodo.com.co/producto/1000591-ibuprofeno-genfar-800-mg-caja-x-50-tabletas`, reducir la ventana a ~800 px y recorrer la página: no aparecen el precio ($17.560) ni el botón. Al ensanchar a 992 px, el bloque aparece.

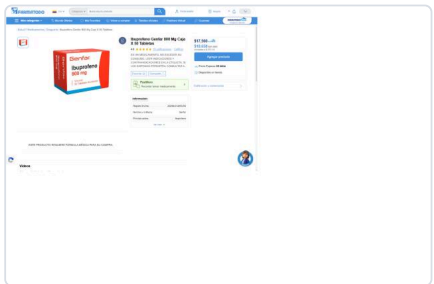
**Causa verificada en el DOM.** El bloque de compra tiene `d-none d-lg-block`: queda oculto por debajo de 992 px.




~800 px: sin precio ni compra




375 px: barra "Agregar producto"



1366 px: bloque de compra visible

**Impacto.** Bloquea la conversión: el usuario llega al producto y no puede comprar, sin ningún indicio de que falte algo.

**Propuesta.** Mostrar un componente de compra en todos los anchos (barra fija por debajo de 992 px) y agregar pruebas de regresión visual en 768, 800, 900 y 991 px.

## UX-FT-002 — Los filtros de búsqueda no se reconocen como seleccionables y no son accesibles

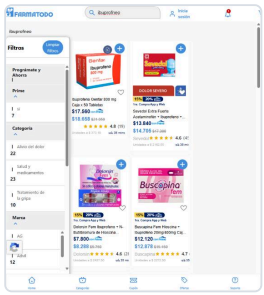
Vista: web desktop (~800 px) · Flujo: búsqueda · Severidad 3 — mayor

**Qué pasa.** El indicador de cada filtro (Categoría, Marca, Prime) se dibuja como una **raya de 2 × 12 px** en lugar de una casilla: no parece un control. Las opciones son `div` sin rol, `tabindex` ni `aria-checked`, así que no se alcanzan con teclado ni las anuncia un lector de pantalla.

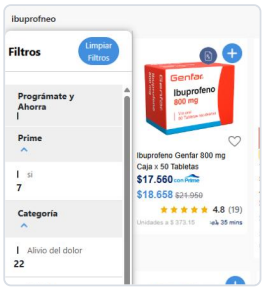
**Cómo replicarlo.** Con la ventana a ~800 px, buscar un producto (por ejemplo `ibuprofneo`) y observar el panel *Filtros*; inspeccionar una opción en DevTools.

**Impacto.** Los filtros parecen texto informativo. Incumple WCAG 2.2 §2.1.1 (Teclado), §4.1.2 (Nombre, función, valor) y §2.5.8 (objetivo mínimo de 24 × 24 px).

**Propuesta.** Usar `input type="checkbox"` nativos con `<label>`, un indicador de al menos 24 × 24 px y estado seleccionado visible.



Resultados a ~800 px



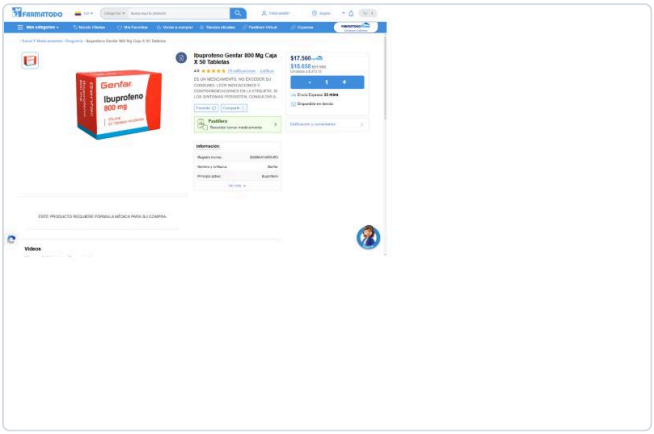

Detalle: rayas en vez de casillas

## UX-FT-003 — El carrito acepta un medicamento con fórmula sin avisar y agrega una bolsa que el usuario no pidió

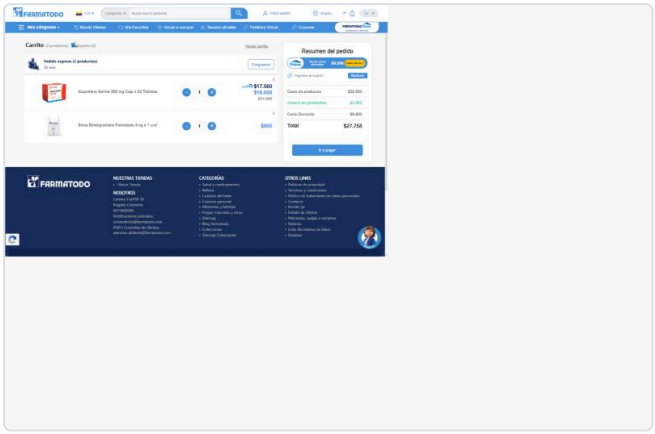
Vista: web desktop (1366 y ~800 px) · Flujo: detalle → carrito → checkout · Severidad 3 — mayor

**Qué pasa.** (1) El ibuprofeno 800 mg requiere fórmula médica, pero al agregarlo no aparece ningún aviso, y en el carrito tampoco. (2) Tras agregar solo el ibuprofeno, el carrito muestra **2 productos**: se suma una "Bolsa Biodegradable Farmatodo" ($600) sin explicación, que desaparece sola al quitar el medicamento.

**Cómo replicarlo.** En la página del producto a 1366 px, presionar *Agregar producto*, abrir `/carrito` y presionar *Ir a pagar* (abre el login: límite del análisis).



Agregado sin aviso de fórmula



Carrito con la bolsa agregada sola

**Impacto.** El usuario descubre tarde que necesita fórmula (pedidos fallidos, contactos a soporte), y un ítem cobrado que no agregó afecta la confianza.

**Propuesta.** Rotular "Requiere fórmula médica" junto al botón de compra y en el carrito, explicando la carga de la fórmula antes del pago. Si la bolsa es obligatoria, indicar el motivo; si no, permitir quitarla.

# Ejercicio 4 — Contador de frecuencia de palabras

**Ver en vivo:** Algoritmo en el portal · Casos TC-ALG · Código

Programa en Python que lee un archivo de texto y muestra la cantidad de palabras, la cantidad de caracteres (incluyendo espacios, sin saltos de línea) y la frecuencia de cada palabra, de mayor a menor y alfabéticamente en caso de empate. Solo usa la biblioteca estándar. Código: `src/farmatodo_qa/word_frequency/`.

### Entrada (ejemplo del PDF)

```
farmatodo farmatodo farmatodo farmatodo farmatodo cuida cuida tu salud
farmatodo cuida toda tu salud
```

### Ejecución

```
uv run word-frequency data/samples/farmatodo_example.txt
```

### Salida (idéntica a la esperada, verificada byte a byte)

```
14 words
99 characters
farmatodo: 6
cuida: 3
salud: 2
tu: 2
toda: 1
```

### Código principal

```
_WORD_RE = re.compile(r"[^\W_]+(?:['’\-][^\W_]+)*")   # letras/dígitos Unicode, con ' o -
internos
_LINE_BREAKS = frozenset("\r\n")

def tokenize(text: str) -> list[str]:
    normalized = unicodedata.normalize("NFC", text)
    return [m.group(0).casefold() for m in _WORD_RE.finditer(normalized)]

def count_characters(text: str) -> int:
    return sum(1 for char in text if char not in _LINE_BREAKS)

def sort_frequencies(counter: Counter[str]) -> tuple[tuple[str, int], ...]:
    return tuple(sorted(counter.items(), key=lambda item: (-item[1], item[0])))

def analyze_lines(lines: Iterable[str]) -> TextStats:
    counter: Counter[str] = Counter()
    words = characters = 0
    for line in lines:                        # streaming: una línea en memoria a la vez
        tokens = tokenize(line)
        words += len(tokens)
        counter.update(tokens)
        characters += count_characters(line)
    return TextStats(words, characters, sort_frequencies(counter))
```

### Algoritmo

1. Leer el archivo línea por línea (UTF-8).
2. En cada línea, contar los caracteres que no son `\n` ni `\r`, extraer las palabras, pasarlas a minúsculas y acumularlas en un diccionario.
3. Ordenar por `(-frecuencia, palabra)`: mayor frecuencia primero y, en empate, orden alfabético.
4. Imprimir el resultado (formato separado del cálculo; `--histogram` agrega barras).

### Complejidad

Con **N** = caracteres de la entrada, **U** = palabras distintas y **L** = largo de la línea más larga:

- **Tiempo: O(N + U log U).** Cada carácter se recorre una vez y cada inserción en el diccionario es O(1) amortizado; ordenar las U palabras cuesta O(U log U). En el peor caso (todas distintas) es O(N log N).
- **Espacio: O(U + L).** El diccionario guarda U palabras y solo hay una línea en memoria.

Un test lo verifica empíricamente: duplicar la entrada (100 000 → 200 000 palabras) aproximadamente duplica el tiempo.

### Decisiones y pruebas

- **Qué es una palabra:** letras o dígitos Unicode, con apóstrofo o guion internos (`don't`, `e-commerce`). La puntuación no cuenta, `Hola` y `hola` son la misma palabra y `camión` no se parte en dos.
- **Desempate:** orden alfabético por punto de código, determinista e independiente de la configuración regional.
- **11 casos automatizados** (TC-ALG-001…011): salida exacta del PDF, conteo de palabras y caracteres, orden y desempate, puntuación y mayúsculas, acentos y ñ, separadores de espacio, archivo vacío, errores de lectura, histograma y escalamiento lineal.
- **Diferencia con WordCounter:** el sitio desempata por orden de aparición (`tu` antes que `salud`); el PDF pide orden alfabético (`salud` antes que `tu`). Ambos coinciden en 14 palabras y 99 caracteres.

# Enfoque de calidad y conclusiones

**Ver en vivo:** SDD · Estrategia · Requisitos · Arquitectura · Decisiones (ADR) · Performance

## Cómo se organizó el trabajo

| Práctica | Qué se hizo |
|---|---|
| Especificación primero | 21 requisitos y 51 casos de prueba en `specs/`, cada uno con requisito, riesgo (impacto × probabilidad), prioridad, pasos y resultado esperado. La matriz de trazabilidad se genera y valida automáticamente. |
| Priorización por riesgo | 9 casos P0 (los tres objetivos de WordCounter, el reporte resumen, el núcleo del algoritmo y el quality gate), P1 para bordes reales y P2 para casos poco frecuentes, performance y defectos conocidos. |
| Arquitectura | Núcleo puro (oráculo, algoritmo, métricas, gate) separado de la infraestructura (Selenium, pytest, disco). La regla se verifica en CI con import-linter. 90 % de cobertura de código del framework. |
| Flaky tests | Cada intento se registra. Pasar al reintentar es FLAKY, nunca PASS; un P0 flaky bloquea. El historial detecta tests inestables entre corridas. |
| CI/CD | GitHub Actions: chequeos en cada PR, suite funcional en cada push y a diario, performance semanal, historial en la rama `qa-history` y portal en GitHub Pages. `master` protegida y `develop` sincronizada. |

## Principales hallazgos

- **WordCounter:** 4 defectos confirmados. El más relevante, BUG-WC-003: Keyword Density no separa palabras unidas por NBSP, algo frecuente al pegar texto.
- **Farmatodo:** 3 hallazgos UX. El más grave, UX-FT-001: entre 768 y 991 px no se puede comprar desde el detalle de producto.

## Alcance

- wordcounter.net y farmatodo.com se prueban como sitios de terceros: una caída se registra como BLOCKED y un cambio de DOM como fallo categorizado.
- El oráculo de WordCounter modela 42 observaciones reales del sitio y se prueba contra ellas.
- La performance es descriptiva y se mide en GitHub Actions; no se fijaron SLA sin datos del negocio.
- El análisis UX cubre búsqueda, detalle de producto y carrito hasta el inicio de sesión del checkout, sin crear cuentas ni comprar.

## Próximos pasos

- Ejecutar la suite E2E también en Firefox (la fábrica de navegadores ya lo admite) y en más resoluciones.
- Regresión visual para los hallazgos responsive de Farmatodo.
- Confirmar con el equipo las preguntas abiertas (desempate con acentos, formato del histograma).

> **Para revisar en vivo:** el portal de calidad muestra los resultados actualizados, la trazabilidad y la tendencia; el repositorio contiene todo el código y las especificaciones, con un README que explica cómo ejecutarlo en una máquina limpia.